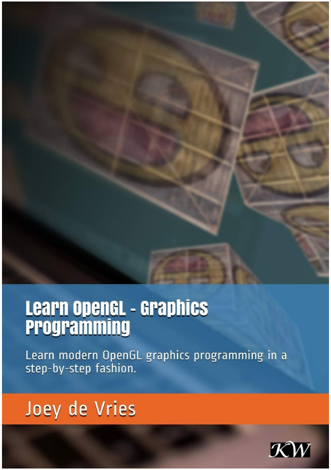
Hello Triangle

Introduction to Computer Graphics - Exercise 02

Pirmin Pfeifer

Resources

The OpenGL exercies are based on the LearnOpenGL course



Tools and Software

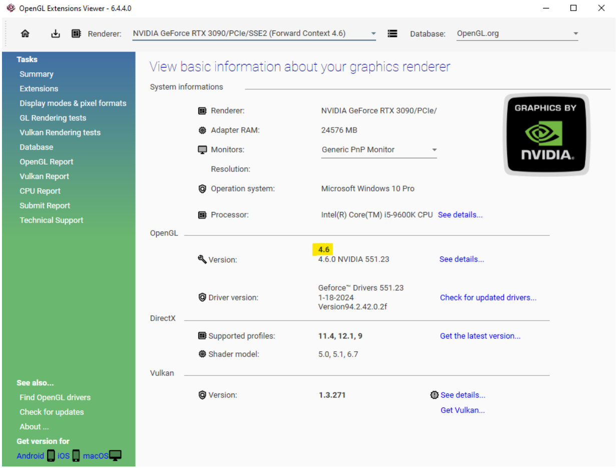
What you need:

- IDE for C++ (Visual Studio)
- CMake
- The provided base project from Moodle
- GPU with support for OpenGL Version 4.3.
  - dedicated GPU
  - modern integrated GPU (Intel HD Graphics or AMD Radeon)
- Git

Ensure the provided base project is compiling on your machine!

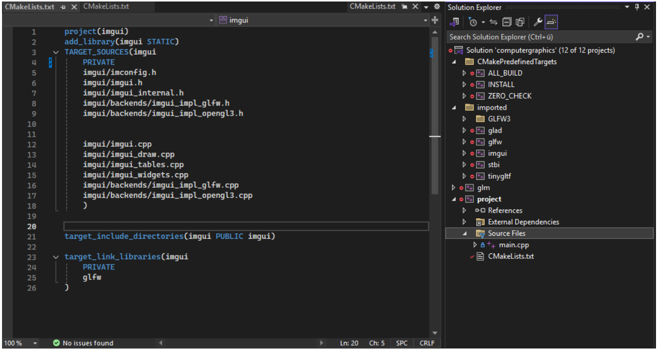
OpenGL Support

OpenGL Version 4.3. or above is recomended.
Check for your OpenGL support with **glxinfo** on Linux
or **OpenGL Extensions Viewer** on Windows.

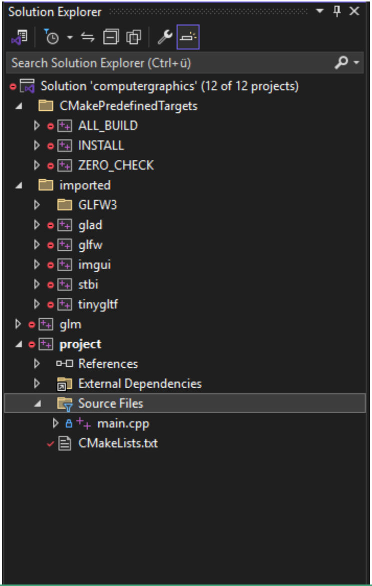


The Project Structure

You can find a basic cmake project with the required libraries already setup on Moodle & Github.

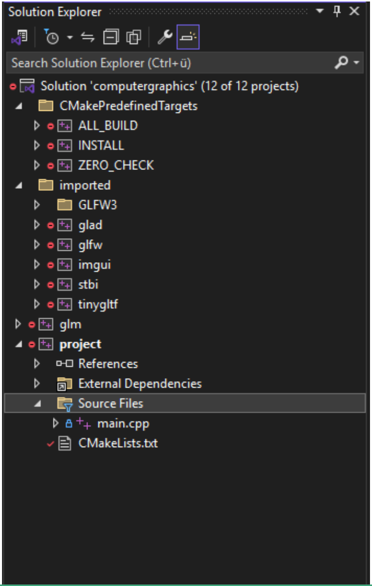


The Project Structure



CmakePredefinedTargets:

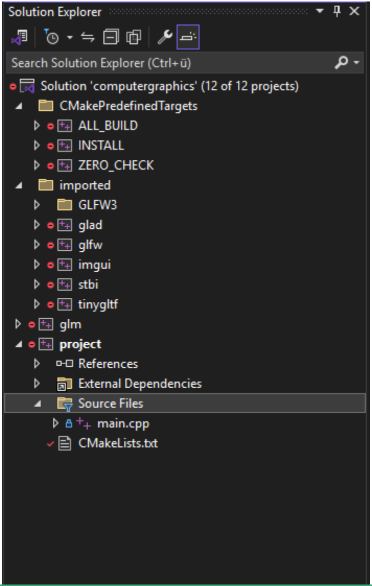
The Project Structure



CmakePredefinedTargets:

- ALL_BUILD → start a build of the whole solution

The Project Structure

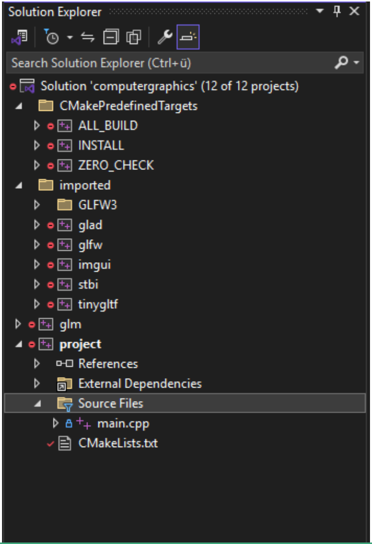


CmakePredefinedTargets:

- ALL_BUILD → start a build of the whole solution
- ZERO_CHECK → reruns cmake configure

The Project Structure

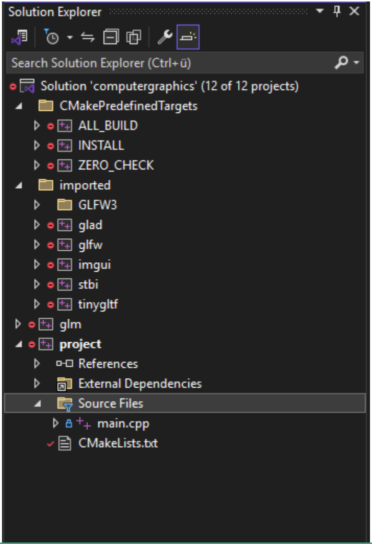
CMake is used to structure the Project



imported 3rd Party Libraries:

The Project Structure

CMake is used to structure the Project

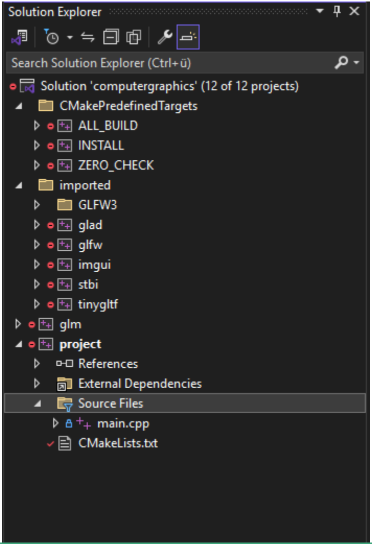


imported 3rd Party Libraries:

- glad - retrieves OS specific OpenGL driver

The Project Structure

CMake is used to structure the Project

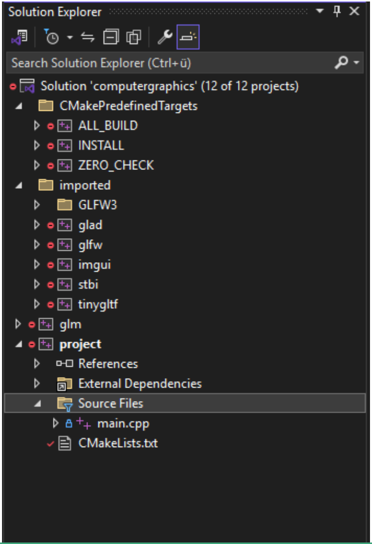


imported 3rd Party Libraries:

- glad - retrieves OS specific OpenGL driver
- glfw - Window Management for OpenGL

The Project Structure

CMake is used to structure the Project

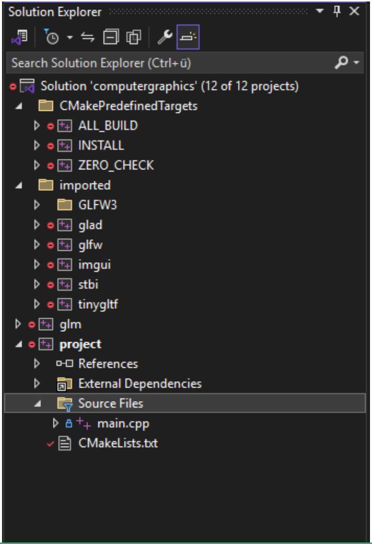


imported 3rd Party Libraries:

- glad - retrieves OS specific OpenGL driver
- glfw - Window Management for OpenGL
- GLFW3 - utilities for glfw

The Project Structure

CMake is used to structure the Project

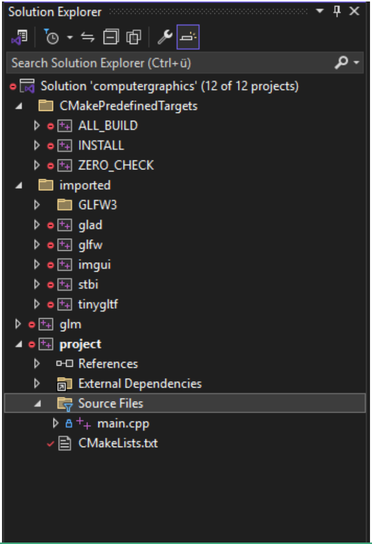


imported 3rd Party Libraries:

- glad - retrieves OS specific OpenGL driver
- glfw - Window Management for OpenGL
- GLFW3 - utilities for glfw
- imgui - common UI Lib

The Project Structure

CMake is used to structure the Project

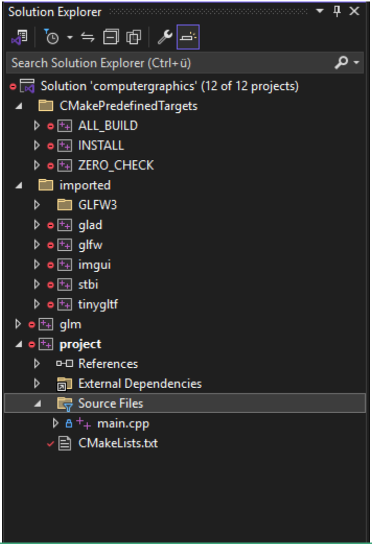


imported 3rd Party Libraries:

- glad - retrieves OS specific OpenGL driver
- glfw - Window Management for OpenGL
- GLFW3 - utilities for glfw
- imgui - common UI Lib
- stbi - standard image lib

The Project Structure

CMake is used to structure the Project



imported 3rd Party Libraries:

- glad - retrieves OS specific OpenGL driver
- glfw - Window Management for OpenGL
- GLFW3 - utilities for glfw
- imgui - common UI Lib
- stbi - standard image lib
- tinygltf - model loader lib

OpenGL

OpenGL is a Graphics API developed by the Khronos Group

OpenGL

OpenGL is a Graphics API developed by the Khronos Group

- Functions and expected results are defined

OpenGL

OpenGL is a Graphics API developed by the Khronos Group

- Functions and expected results are defined
  $\Rightarrow$ implemented by the graphics driver

OpenGL

OpenGL is a Graphics API developed by the Khronos Group

- Functions and expected results are defined
  $\Rightarrow$ implemented by the graphics driver
- It provides hardware vendor specific extensions

OpenGL

OpenGL is a Graphics API developed by the Khronos Group

- Functions and expected results are defined
  $\Rightarrow$ implemented by the graphics driver
- It provides hardware vendor specific extensions
- Works on an internal global state machine

OpenGL

Duale Hochschule Baden-Württemberg

OpenGL is a Graphics API developed by the Khronos Group

- Functions and expected results are defined
  $\Rightarrow$ implemented by the graphics driver
- It provides hardware vendor specific extensions
- Works on an internal global state machine
- Is effectively an C-library

OpenGL

OpenGL is a Graphics API developed by the Khronos Group

- Functions and expected results are defined
  $\Rightarrow$ implemented by the graphics driver
- It provides hardware vendor specific extensions
- Works on an internal global state machine
- Is effectively an C-library
  $\Rightarrow$ no object orientation

Why OpenGL

What are the alternatives:

Why OpenGL

What are the alternatives:

Game Engines

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs
- too capable

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs
- too capable

D3D12

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs
- too capable

D3D12

PROS:

- very close to Hardware

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs
- too capable

D3D12

PROS:

- very close to Hardware

CONS:

- Windows only

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs
- too capable

D3D12

PROS:

- very close to Hardware

CONS:

- Windows only
- too steep learning curve

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs
- too capable

D3D12

PROS:

- very close to Hardware

CONS:

- Windows only
- too steep learning curve
- LOC for a Triangle > 1000

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs
- too capable

D3D12

PROS:

- very close to Hardware

CONS:

- Windows only
- too steep learning curve
- LOC for a Triangle > 1000

Vulkan

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs
- too capable

D3D12

PROS:

- very close to Hardware

CONS:

- Windows only
- too steep learning curve
- LOC for a Triangle > 1000

Vulkan

PROS:

- very close to Hardware

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs
- too capable

D3D12

PROS:

- very close to Hardware

CONS:

- Windows only
- too steep learning curve
- LOC for a Triangle > 1000

Vulkan

PROS:

- very close to Hardware

CONS:

- too steep learning curve

Why OpenGL

What are the alternatives:

Game Engines

PROS:

- easy to learn
- very capable

CONS:

- hides the graphics APIs
- too capable

D3D12

PROS:

- very close to Hardware

CONS:

- Windows only
- too steep learning curve
- LOC for a Triangle > 1000

Vulkan

PROS:

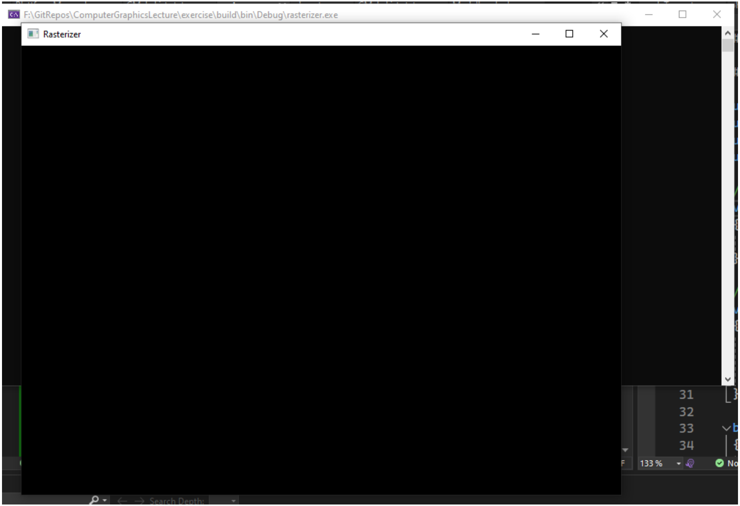
- very close to Hardware

CONS:

- too steep learning curve
- even more LOC for a Triangle

Hello Window!

Lets start with creating a Window:



Hello Window!

What do we need[1]:

Hello Window!

What do we need[1]:

- glad - OpenGL Context & Function bindings

[1] The base project should compile and the compatibilityCheck should run with out error

Hello Window!

What do we need[1]:

- glad - OpenGL Context & Function bindings
- GLFW - Platform independent Window abstractions

[1] The base project should compile and the compatibilityCheck should run with out error

Hello Window!

What do we need[1]:

- glad - OpenGL Context & Function bindings
- GLFW - Platform independent Window abstractions

```cpp
// glad for OPGL context
#include <glad/glad.h>
// GLFW for window management
#include <GLFW/glfw3.h>
```

File: main.cpp

[1] The base project should compile and the compatibilityCheck should run with out error

Hello Window! - Steps

To create get a window we need:

Hello Window! - Steps

To create get a window we need:

1. Init GLFW Library and set opengl version and extension hints

Hello Window! - Steps

To create get a window we need:

1. Init GLFW Library and set opengl version and extension hints
2. Create a Window

Hello Window! - Steps

To create get a window we need:

1. Init GLFW Library and set opengl version and extension hints
2. Create a Window
3. Set created window to current window in glfw context

Hello Window! - Steps

To create get a window we need:

1. Init GLFW Library and set opengl version and extension hints
2. Create a Window
3. Set created window to current window in glfw context
4. Use glad to load opengl function pointers
5. Setup open gl view port

Hello Window! - Steps

To create get a window we need:

1. Init GLFW Library and set opengl version and extension hints
2. Create a Window
3. Set created window to current window in glfw context
4. Use glad to load opengl function pointers
5. Setup open gl view port
6. Setup resize callback

Hello Window! - Steps

To create get a window we need:

1. Init GLFW Library and set opengl version and extension hints
2. Create a Window
3. Set created window to current window in glfw context
4. Use glad to load opengl function pointers
5. Setup open gl view port
6. Setup resize callback
7. Add Render & Update Loop

Hello Window! - Steps

To create get a window we need:

1. Init GLFW Library and set opengl version and extension hints
2. Create a Window
3. Set created window to current window in glfw context
4. Use glad to load opengl function pointers
5. Setup open gl view port
6. Setup resize callback
7. Add Render & Update Loop
8. Clean up after loop

Hello Window! - Render Loop

The render loop is used to update the image displayed in the window:

Hello Window! - Render Loop

The render loop is used to update the image displayed in the window:

1. Handle Input

Hello Window! - Render Loop

The render loop is used to update the image displayed in the window:

1. Handle Input
2. Clear the canvas/framebuffer
3. Render new image (at first this will just be setting a flat color)

Hello Window! - Render Loop

The render loop is used to update the image displayed in the window:

1. Handle Input
2. Clear the canvas/framebuffer
3. Render new image (at first this will just be setting a flat color)
4. Swap buffers to present the rendered image

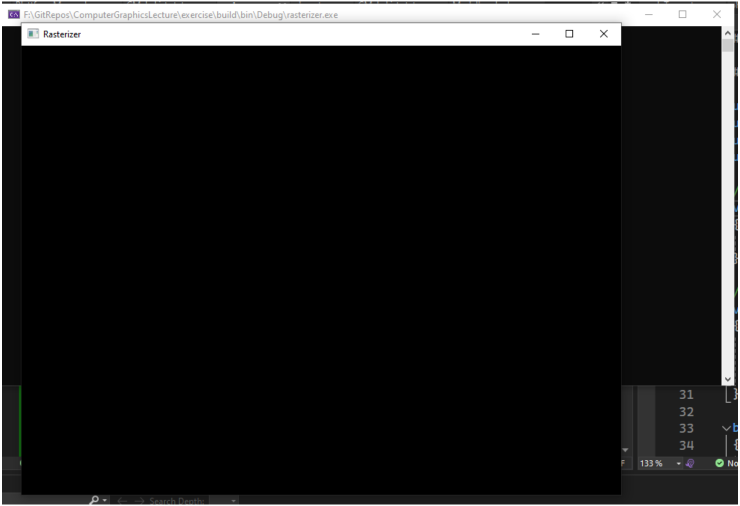
Hello Window! - Render Loop

The render loop is used to update the image displayed in the window:

1. Handle Input
2. Clear the canvas/framebuffer
3. Render new image (at first this will just be setting a flat color)
4. Swap buffers to present the rendered image
5. pool events to recive updates such as key down event or resizes

Hello Window!

Now we have a nice black window:



Hello Input! Bye Window!

Close on escape key pressed:

```cpp
while (!glfwWindowShouldClose(window))
{
    if(glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS) {
        glfwSetWindowShouldClose(window, true);
    }

    glfwSwapBuffers(window);
    glfwPollEvents();
}
```

File: main.cpp (render loop)

Hello Color!

In the render loop we can clear the buffer using a ClearColor:

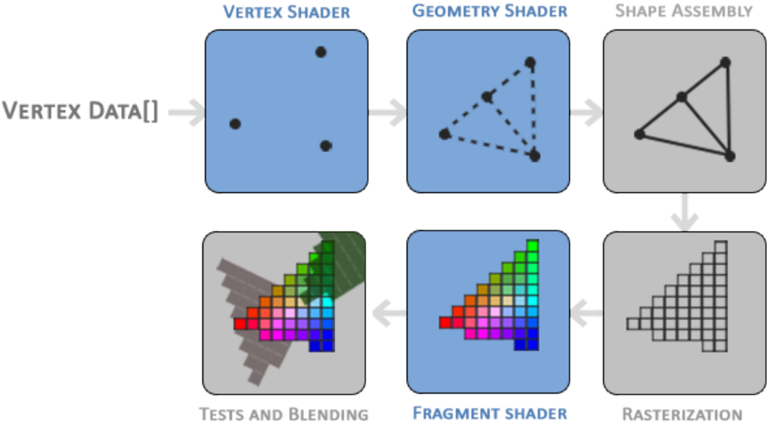
```
glClearColor(0.2f, 0.3f, 0.3f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);
```

File: main.cpp (updated render loop)

Now it is your turn!

Use LearnOpenGL to create a window!

Hello Triangle!



Hello Triangle! - Vertex Input

Vertex Coordinates in a array:

```cpp
float vertices[] = {
    -0.5f, -0.5f, 0.0f,
     0.5f, -0.5f, 0.0f,
     0.0f,  0.5f, 0.0f
};
```

File: main.cpp

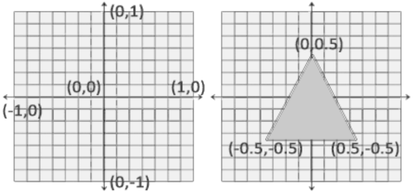
Hello Triangle! - Vertex Input

DHBW
Duale Hochschule
Baden-Württemberg

Vertex Coordinates in a array:

```
float vertices[] = {
    -0.5f, -0.5f, 0.0f,
     0.5f, -0.5f, 0.0f,
     0.0f,  0.5f, 0.0f
};
```

File: main.cpp Normalized Device Coordinates (NDC):

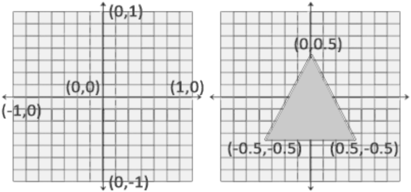


Hello Triangle! - Vertex Input

Vertex Coordinates in a array:

```
float vertices[] = {
    -0.5f, -0.5f, 0.0f,
    0.5f, -0.5f, 0.0f,
    0.0f,  0.5f, 0.0f
};
```

File: main.cpp Normalized Device Coordinates (NDC):



NCD will be transformed into view space coordinates using the specified viewport

Hello Triangle! - Vertex Input

We need to upload our vertices to the GPU before we can render them:

File: main.cpp

Hello Triangle! - Vertex Input

We need to upload our vertices to the GPU before we can render them:

1. Create buffer

```cpp
// Generate a Vertex Buffer Object
unsigned int VBO;
glGenBuffers(1, &VBO);
```

File: main.cpp

Hello Triangle! - Vertex Input

We need to upload our vertices to the GPU before we can render them:

1. Create buffer

```
// Generate a Vertex Buffer Object
unsigned int VBO;
glGenBuffers(1, &VBO);
```

2. Bind buffer

```
glBindBuffer(GL_ARRAY_BUFFER, VBO);
```

File: main.cpp

Hello Triangle! - Vertex Input

We need to upload our vertices to the GPU before we can render them:

1. Create buffer

```
// Generate a Vertex Buffer Object
unsigned int VBO;
glGenBuffers(1, &VBO);
```

2. Bind buffer

```
glBindBuffer(GL_ARRAY_BUFFER, VBO);
```

3. Upload data to buffer

```
glBufferData(
    GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

File: main.cpp

Hello Triangle! - Vertex Shader

```glsl
#version 330 core
layout (location = 0) in vec3 aPos;

void main()
{
    gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0);
}
```

Vertex Shader - Load and Compile

Create a vertex shader:

```
unsigned int vertexShader;
vertexShader = glCreateShader(GL_VERTEX_SHADER);
```

Vertex Shader - Load and Compile

Create a vertex shader:

```
unsigned int vertexShader;
vertexShader = glCreateShader(GL_VERTEX_SHADER);
```

Compile the shader source:

```
const char* vertexShaderSourcePtr = vertexShaderSource.c_str();
glShaderSource(vertexShader, 1, &vertexShaderSourcePtr, NULL);
glCompileShader(vertexShader);
```

File: main.cpp

Vertex Shader - Error Checking

We probably should check if the compilation was successful:

```cpp
int  success;
char infoLog[512];
glGetShaderiv(vertexShader, GL_COMPILE_STATUS, &success);
if(!success)
{
    glGetShaderInfoLog(vertexShader, 512, NULL, infoLog);
    std::cout << "ERROR::SHADER::VERTEX::COMPILATION_FAILED\n"
              << infoLog
              << std::endl;
    return -1;
}
```

File: main.cpp

Fragment Shader

Duale Hochschule Baden-Württemberg

```
#version 330 core
out vec4 FragColor;

void main()
{
    FragColor = vec4(1.0f, 0.5f, 0.2f, 1.0f);
}
```

Fragment Shader - Load and Compile

Create a fragment shader:

```
unsigned int fragmentShader;
fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
```


Fragment Shader - Load and Compile

Create a fragment shader:

```
unsigned int fragmentShader;
fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
```

Compile the shader source:

```
const char* fragmentShaderSourcePtr = fragmentShaderSource.c_str();
glShaderSource(fragmentShader, 1, &fragmentShaderSourcePtr, NULL);
glCompileShader(fragmentShader);
```

Fragment Shader - Load and Compile

Create a fragment shader:

```
unsigned int fragmentShader;
fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
```

Compile the shader source:

```
const char* fragmentShaderSourcePtr = fragmentShaderSource.c_str();
glShaderSource(fragmentShader, 1, &fragmentShaderSourcePtr, NULL);
glCompileShader(fragmentShader);
```

Error checking is the same as with the vertex shader.
File: main.cpp

Shader Program

Now we combine the two shaders to a Shader Program:

Shader Program

Now we combine the two shaders to a Shader Program:

```
// create new shader program
unsigned int shaderProgram;
shaderProgram = glCreateProgram();
// link shader program
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
```

Shader Program

Now we combine the two shaders to a Shader Program:

```
// create new shader program
unsigned int shaderProgram;
shaderProgram = glCreateProgram();
// link shader program
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
```

With `glGetProgramiv` and `glGetProgramInfoLog` we can check for an error in the link process.

Shader Program

Now we combine the two shaders to a Shader Program:

```
// create new shader program
unsigned int shaderProgram;
shaderProgram = glCreateProgram();
// link shader program
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
```

With `glGetProgramiv` and `glGetProgramInfoLog` we can check for an error in the link process.

With `glUseProgram( shaderProgram )` we can bind the shader program to the state machine. Every draw call afterwards uses the shader.

Shader Program

Now we combine the two shaders to a Shader Program:

```
// create new shader program
unsigned int shaderProgram;
shaderProgram = glCreateProgram();
// link shader program
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
```

With `glGetProgramiv` and `glGetProgramInfoLog` we can check for an error in the link process.

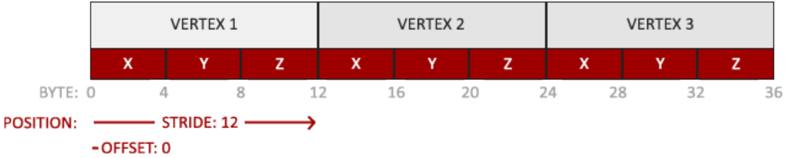
With `glUseProgram( shaderProgram )` we can bind the shader program to the state machine. Every draw call afterwards uses the shader.

At the end of our program we should release the shaders with `glDeleteShader( shaderId )`.

File: main.cpp

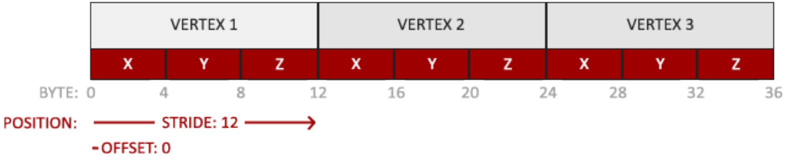
Linking Vertex Attributes

Lets have another look at our vertex buffer:



Linking Vertex Attributes

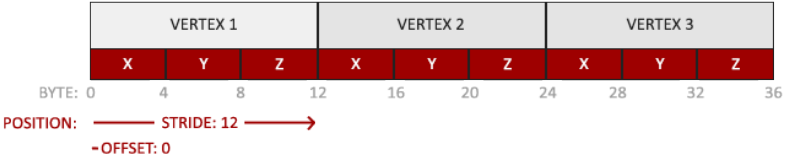
Lets have another look at our vertex buffer:



- The position data is stored as 32-bit (4 byte) floating point values.

Linking Vertex Attributes

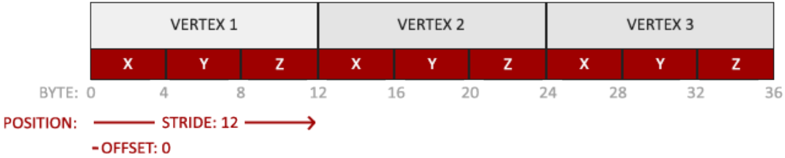
Lets have another look at our vertex buffer:



- The position data is stored as 32-bit (4 byte) floating point values.
- Each position is composed of 3 of those values.

Linking Vertex Attributes

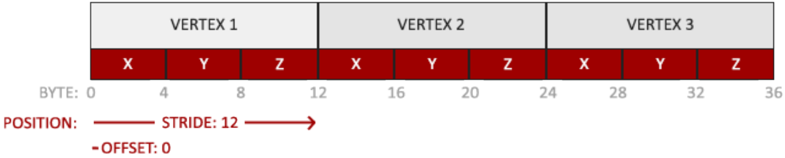
Lets have another look at our vertex buffer:



- The position data is stored as 32-bit (4 byte) floating point values.
- Each position is composed of 3 of those values.
- There is no space (or other values) between each set of 3 values. The values are tightly packed in the array.

Linking Vertex Attributes

Lets have another look at our vertex buffer:



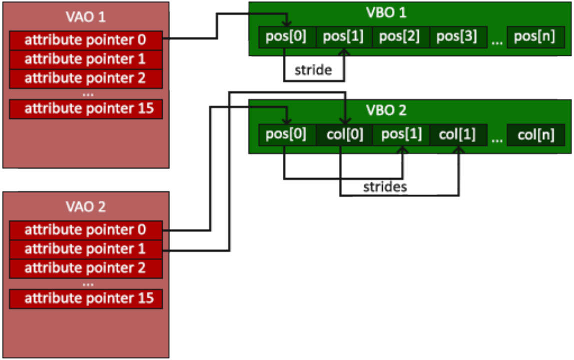
- The position data is stored as 32-bit (4 byte) floating point values.
- Each position is composed of 3 of those values.
- There is no space (or other values) between each set of 3 values. The values are tightly packed in the array.
- The first value in the data is at the beginning of the buffer.

Vertex Array Object

We need to tell the GPU where in our buffer the vertices are and how they are aligned.

Vertex Array Object

We need to tell the GPU where in our buffer the vertices are and how they are aligned.



We use a Vertex Array Object to provide the Vertex Attributes Information.

Vertex Array Object

So we need to create a Vertex Array Object (VAO):

Vertex Array Object

So we need to create a Vertex Array Object (VAO):

```
unsigned int VAO;
glGenVertexArrays(1, &VAO);
```

Vertex Array Object

So we need to create a Vertex Array Object (VAO):

```
unsigned int VAO;
glGenVertexArrays(1, &VAO);
```

Bind buffers and upload vertices[2]:

Vertex Array Object

So we need to create a Vertex Array Object (VAO):

```
unsigned int VAO;
glGenVertexArrays(1, &VAO);
```

Bind buffers and upload vertices[2]:

```
glBindVertexArray(VAO);
// 2. copy our vertices array in a buffer for OpenGL to use
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(
    GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

Vertex Array Object

So we need to create a Vertex Array Object (VAO):

```
unsigned int VAO;
glGenVertexArrays(1, &VAO);
```

Bind buffers and upload vertices[2]:

```
glBindVertexArray(VAO);
// 2. copy our vertices array in a buffer for OpenGL to use
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(
    GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

[2] we only need to do this once, but we need to bind the buffer here so we can combine both

Vertex Array Object

So we need to create a Vertex Array Object (VAO):

```
unsigned int VAO;
glGenVertexArrays(1, &VAO);
```

Bind buffers and upload vertices[2]:

```
glBindVertexArray(VAO);
// 2. copy our vertices array in a buffer for OpenGL to use
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(
    GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

Set up the Vertex Attribute Pointer:

[2] we only need to do this once, but we need to bind the buffer here so we can combine both

Vertex Array Object

So we need to create a Vertex Array Object (VAO):

```
unsigned int VAO;
glGenVertexArrays(1, &VAO);
```

Bind buffers and upload vertices[2]:

```
glBindVertexArray(VAO);
// 2. copy our vertices array in a buffer for OpenGL to use
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(
    GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

Set up the Vertex Attribute Pointer:

```
// 3. then set our vertex attributes pointers
glVertexAttribPointer(
    0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
```

File: main.cpp

[2] we only need to do this once, but we need to bind the buffer here so we can combine both

Vertex Array Object

Lets have a closer look at: `glVertexAttribPointer (`

```
)
```

```
glVertexAttribPointer (  ,  ,           ,           ,                ,        );
```

Vertex Array Object

DHBW Duale Hochschule Baden-Württemberg

Lets have a closer look at: `glVertexAttribPointer (`

- `GLuint index` index of the attribute in the shader

`)`

```
glVertexAttribPointer ( 0,  ,          ,          ,               ,        );
```

Vertex Array Object

Lets have a closer look at: `glVertexAttribPointer (`

- `GLuint index` index of the attribute in the shader
- `GLuint size` number of components per vertex

```
)
```

```
glVertexAttribPointer ( 0, 3,            ,            ,                 ,          );
```

Vertex Array Object

Lets have a closer look at: `glVertexAttribPointer (`

- `GLuint index` index of the attribute in the shader
- `GLuint size` number of components per vertex
- `GLenum type` type of data (GL_FLOAT,...)

`)`

```
glVertexAttribPointer ( 0, 3,  GL_FLOAT ,              ,                    ,         );
```

Vertex Array Object

Lets have a closer look at: `glVertexAttribPointer (`

- `GLuint index` index of the attribute in the shader
- `GLuint size` number of components per vertex
- `GLenum type` type of data (GL_FLOAT,. . .)
- `GLboolean normalized` specifies if the data should be normalized

`)`

```
glVertexAttribPointer ( 0, 3, GL_FLOAT , GL_FALSE ,                    ,           );
```

Vertex Array Object

Lets have a closer look at: `glVertexAttribPointer (`

- `GLuint index` index of the attribute in the shader
- `GLuint size` number of components per vertex
- `GLenum type` type of data (GL_FLOAT,. . .)
- `GLboolean normalized` specifies if the data should be normalized
- `GLsizei stride` byte offset between consecutive vertex attributes

`)`

```
glVertexAttribPointer ( 0, 3, GL_FLOAT , GL_FALSE , 3 * sizeof ( float ),          );
```

Vertex Array Object

Lets have a closer look at: `glVertexAttribPointer (`

- `GLuint index` index of the attribute in the shader
- `GLuint size` number of components per vertex
- `GLenum type` type of data (GL_FLOAT,...)
- `GLboolean normalized` specifies if the data should be normalized
- `GLsizei stride` byte offset between consecutive vertex attributes
- `const GLvoid * pointer` offset of the first component in the array

`)`

```
glVertexAttribPointer ( 0, 3, GL_FLOAT , GL_FALSE , 3 * sizeof ( float ), ( void *)0);
```

The final step

The final steps:

The final step

The final steps:

1. bind the shader program before the render loop

```
glUseProgram(shaderProgram);
```

The final step

The final steps:

1. bind the shader program before the render loop

```
glUseProgram(shaderProgram);
```

2. bind the vertex array object

```
glBindVertexArray(VAO);
```

The final step

The final steps:

1. bind the shader program before the render loop

```
glUseProgram(shaderProgram);
```

2. bind the vertex array object

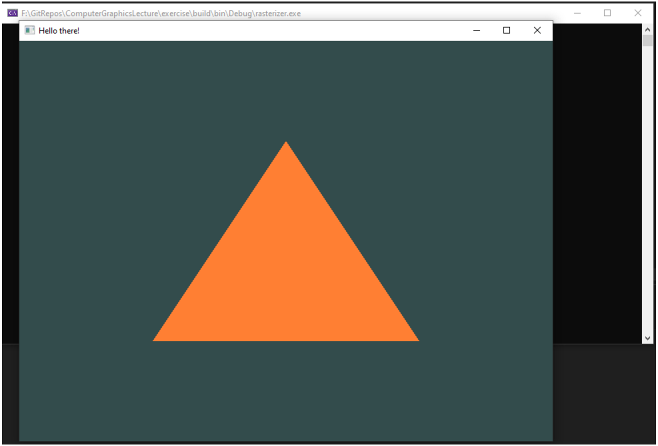
```
glBindVertexArray(VAO);
```

3. make our first draw call

```
glDrawArrays(GL_TRIANGLES, 0, 3);
```

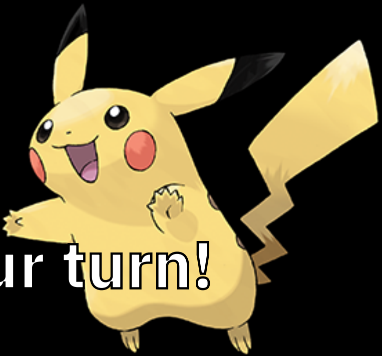
File: main.cpp

Hello There!

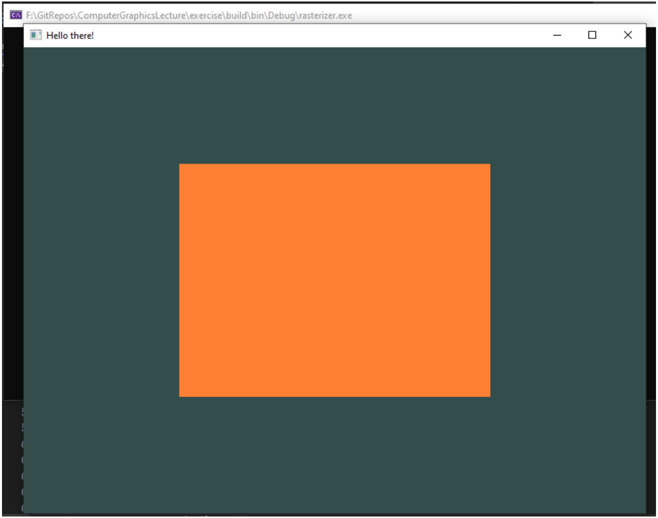


F:\GitRepos\ComputerGraphicsLecture\exercise\build\bin\Debug\rasterizer.exe
Hello there!

Now it is your turn!



Let’s think bigger!



Let’s think bigger!

Lets look at our vertex buffer for the quad:

```cpp
float vertices[] = {
    // first triangle
    0.5f,  0.5f, 0.0f,  // top right
    0.5f, -0.5f, 0.0f,  // bottom right
    -0.5f,  0.5f, 0.0f,  // top left
    // second triangle
    0.5f, -0.5f, 0.0f,  // bottom right
    -0.5f, -0.5f, 0.0f,  // bottom left
    -0.5f,  0.5f, 0.0f   // top left
};
```

File: main.cpp

Let’s think bigger!

Lets look at our vertex buffer for the quad:

```cpp
float vertices[] = {
    // first triangle
    0.5f,  0.5f, 0.0f,  // top right
    0.5f, -0.5f, 0.0f,  // bottom right
    -0.5f,  0.5f, 0.0f,  // top left
    // second triangle
    0.5f, -0.5f, 0.0f,  // bottom right
    -0.5f, -0.5f, 0.0f,  // bottom left
    -0.5f,  0.5f, 0.0f   // top left
};
```

File: main.cpp

Let's think smaller!

We can split our geometry info into vertex and index info:

```cpp
float vertices[] = {
     0.5f,  0.5f, 0.0f,  // top right
     0.5f, -0.5f, 0.0f,  // bottom right
    -0.5f, -0.5f, 0.0f,  // bottom left
    -0.5f,  0.5f, 0.0f   // top left
};
unsigned int indices[] = {  // note that we start from 0!
    0, 1, 3,   // first triangle
    1, 2, 3    // second triangle
};
```

File: main.cpp

Let’s think smaller!

We can split our geometry info into vertex and index info:

```cpp
float vertices[] = {
    0.5f,  0.5f, 0.0f,  // top right
    0.5f, -0.5f, 0.0f,  // bottom right
    -0.5f, -0.5f, 0.0f,  // bottom left
    -0.5f,  0.5f, 0.0f   // top left
};
unsigned int indices[] = {  // note that we start from 0!
    0, 1, 3,   // first triangle
    1, 2, 3    // second triangle
};
```

File: main.cpp

We reduced our memory cost:

Let's think smaller!

DHBW Duale Hochschule Baden-Württemberg

We can split our geometry info into vertex and index info:

```cpp
float vertices[] = {
     0.5f,  0.5f, 0.0f,  // top right
     0.5f, -0.5f, 0.0f,  // bottom right
    -0.5f, -0.5f, 0.0f,  // bottom left
    -0.5f,  0.5f, 0.0f   // top left
};
unsigned int indices[] = {  // note that we start from 0!
    0, 1, 3,   // first triangle
    1, 2, 3    // second triangle
};
```

File: main.cpp

We reduced our memory cost:

$$4b * 3c * 6v = 72 bytes \Rightarrow ?b * ?c * ?v + ?b * ?c * ?t = ??bytes$$

b :=bytes, c :=components , v :=vertices, t :=triangles

Let's think smaller!

We can split our geometry info into vertex and index info:

```cpp
float vertices[] = {
    0.5f,  0.5f, 0.0f,  // top right
    0.5f, -0.5f, 0.0f,  // bottom right
    -0.5f, -0.5f, 0.0f,  // bottom left
    -0.5f,  0.5f, 0.0f   // top left
};
unsigned int indices[] = {  // note that we start from 0!
    0, 1, 3,   // first triangle
    1, 2, 3    // second triangle
};
```

File: main.cpp

We reduced our memory cost:

$$4b * 3c * 6v = 72bytes \Rightarrow 4b * 3c * 4v + ?b * ?c * ?t = ??bytes$$

b :=bytes, c :=components , v :=vertices, t :=triangles

Let's think smaller!

We can split our geometry info into vertex and index info:

```cpp
float vertices[] = {
     0.5f,  0.5f, 0.0f,  // top right
     0.5f, -0.5f, 0.0f,  // bottom right
    -0.5f, -0.5f, 0.0f,  // bottom left
    -0.5f,  0.5f, 0.0f   // top left
};
unsigned int indices[] = {  // note that we start from 0!
    0, 1, 3,   // first triangle
    1, 2, 3    // second triangle
};
```

File: main.cpp

We reduced our memory cost:

$$4b * 3c * 6v = 72bytes \Rightarrow 4b * 3c * 4v + 4b * 3c * 2t = ??bytes$$

b :=bytes, c :=components , v :=vertices, t :=triangles

Let's think smaller!

We can split our geometry info into vertex and index info:

```
float vertices[] = {
    0.5f,  0.5f, 0.0f,  // top right
    0.5f, -0.5f, 0.0f,  // bottom right
   -0.5f, -0.5f, 0.0f,  // bottom left
   -0.5f,  0.5f, 0.0f   // top left
};
unsigned int indices[] = {  // note that we start from 0!
    0, 1, 3,   // first triangle
    1, 2, 3    // second triangle
};
```

File: main.cpp

We reduced our memory cost:

$$4b * 3c * 6v = 72bytes \Rightarrow 4b * 3c * 4v + 4b * 3c * 2t = 72bytes$$

b :=bytes, c :=components , v :=vertices, t :=triangles

Which is nothing at the moment, but we generally add more vertex attributes.

Index Buffer

Allocate Element Buffer Object (index buffer):

```
unsigned int EBO;
glGenBuffers(1, &EBO);
```

Index Buffer

Allocate Element Buffer Object (index buffer):

```
unsigned int EBO;
glGenBuffers(1, &EBO);
```

Bind and upload buffer:

```
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER,
             sizeof(indices), indices,
             GL_STATIC_DRAW);
```

Index Buffer

Allocate Element Buffer Object (index buffer):

```
unsigned int EBO;
glGenBuffers(1, &EBO);
```

Bind and upload buffer:

```
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER,
             sizeof(indices), indices,
             GL_STATIC_DRAW);
```

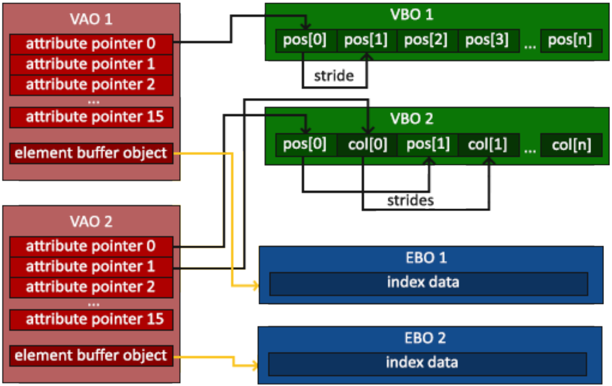
In render loop we now also need to bind the index buffer and use a different draw call:

```
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);
```

File: main.cpp

Index Buffer

The Element Buffer Object is also bound into the Vertex Array Object:



Index Buffer

Our Vertex Array Object inti code look now like this:

```
// 1. bind Vertex Array Object
glBindVertexArray(VAO);
// 2. copy our vertices array in a vertex buffer for OpenGL to use
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER,
             sizeof(vertices), vertices,
             GL_STATIC_DRAW);
// 3. copy our index array in a element buffer for OpenGL to use
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices),
             indices,
             GL_STATIC_DRAW);
// 4. then set the vertex attributes pointers
glVertexAttribPointer(
    0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
```

File: main.cpp

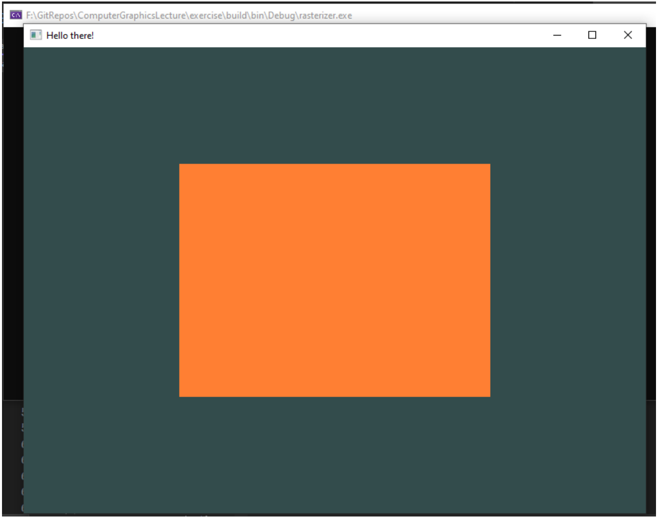
Index Buffer

Finally we update our render loop to something like this:

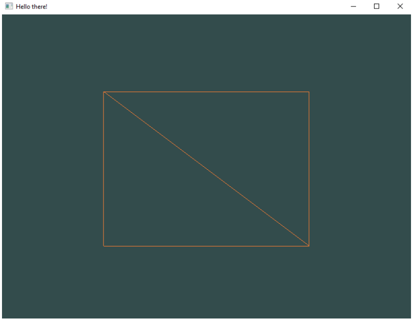
```
glUseProgram(shaderProgram);
while (!glfwWindowShouldClose(window)) {
    // input
    processInput(window);

    // render
    glClearColor(0.2f, 0.3f, 0.3f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glBindVertexArray(VAO);
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);

    // present frame and update events
    glfwSwapBuffers(window);
    glfwPollEvents();
}
```

Index Buffer

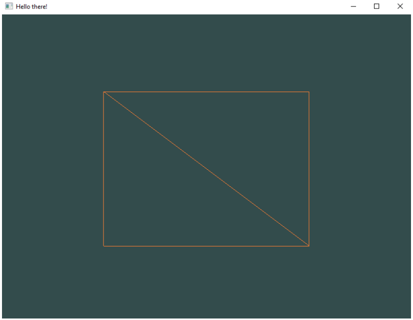
Now we have the same quad, but rendered with an index buffer:



Bonus: Wireframe



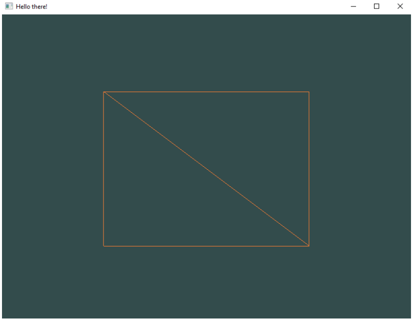
Bonus: Wireframe



We can enable Wireframe mode with:

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)
```

Bonus: Wireframe



File: main.cpp (render loop)

We can enable Wireframe mode with:

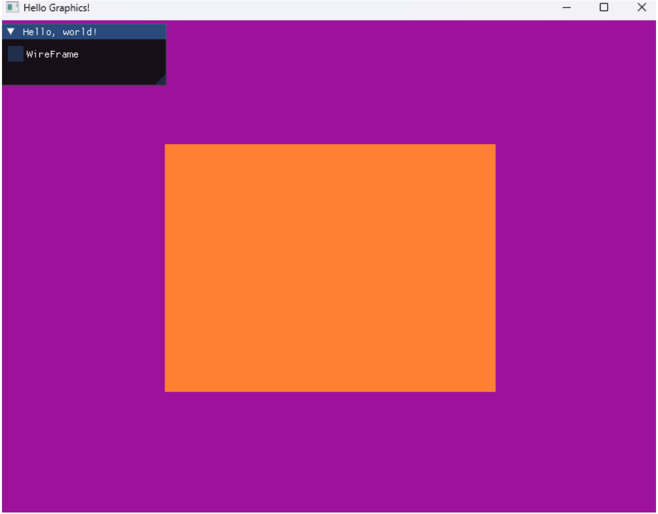
```
glPolygonMode ( GL_FRONT_AND_BACK , GL_LINE )
```

We can disable it again with:

```
glPolygonMode ( GL_FRONT_AND_BACK , GL_FILL )
```

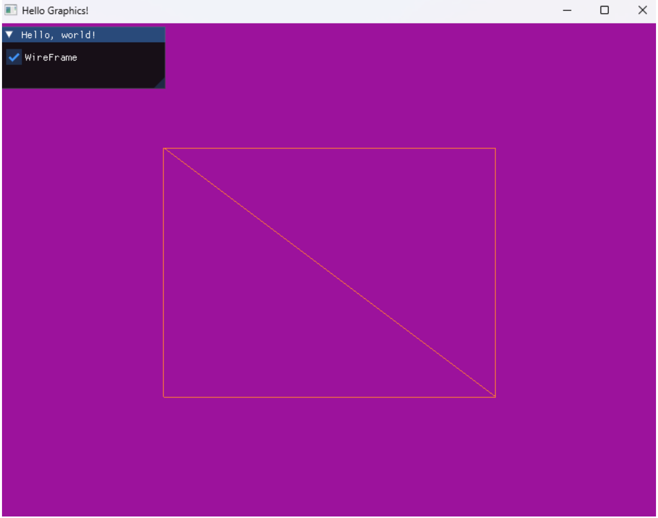
Bonus: Wireframe

We can create a toggle for the wireframe with ImGui:



Bonus: Wireframe

We can create a toggle for the wireframe with ImGui:



Bonus: ImGui

Requiered Includes:

```
#include "imgui.h"
#include "backends/imgui_impl_glfw.h"
#include "backends/imgui_impl_opengl3.h"
```

File: main.cpp

Bonus: ImGui

Requiered Includes:

```
#include "imgui.h"
#include "backends/imgui_impl_glfw.h"
#include "backends/imgui_impl_opengl3.h"
```

To use ImGui we need to setup imgui with opengl:

```
// Setup Dear ImGui context
IMGUI_CHECKVERSION();
ImGui::CreateContext();
ImGuiIO& io = ImGui::GetIO();
io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard;
io.ConfigFlags |= ImGuiConfigFlags_NavEnableGamepad;

// Setup Platform/Renderer backends
ImGui_ImplGlfw_InitForOpenGL(window, true);
ImGui_ImplOpenGL3_Init();
```

File: main.cpp

Bonus: ImGui

Inside the render loop we need to prepere the next frame for ImGui:

```
ImGui_ImplOpenGL3_NewFrame();
ImGui_ImplGlfw_NewFrame();
ImGui::NewFrame();
```

File: main.cpp

Bonus: ImGui

Inside the render loop we need to prepere the next frame for ImGui:

```
ImGui_ImplOpenGL3_NewFrame();
ImGui_ImplGlfw_NewFrame();
ImGui::NewFrame();
```

Afterwards we can create our UI Logic:

```
ImGui::Begin("Hello, world!");
ImGui::SetWindowSize(ImVec2(200, 75), ImGuiCond_Once);
ImGui::Checkbox("WireFrame", &wireframe);
ImGui::End();
```

File: main.cpp

Bonus: ImGui

Inside the render loop we need to prepere the next frame for ImGui:

```
ImGui_ImplOpenGL3_NewFrame();
ImGui_ImplGlfw_NewFrame();
ImGui::NewFrame();
```

Afterwards we can create our UI Logic:

```
ImGui::Begin("Hello, world!");
ImGui::SetWindowSize(ImVec2(200, 75), ImGuiCond_Once);
ImGui::Checkbox("WireFrame", &wireframe);
ImGui::End();
```

At the end, but before swaping the buffers, issue the ImGui Rendering:

```
ImGui::Render();
ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());
```

File: main.cpp

Now it is your turn again!